

Synthesizability lint

Not every Verilog fragment that simulates cleanly will synthesize to real gates

What the linter catches

- The teaching rule pack covers the most common classroom mistakes
- Hash-delay on assigns or gates is simulation timing, not hardware
- Initial blocks and dollar-display tasks belong in testbenches, not RTL modules
- An always-at-star if without else can infer a latch
- Inside an always-ff clocked block, use non-blocking assignment for register outputs
- Inside always-comb, use blocking assignment instead

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Synth lint

INTERACTIVE TOOL

Synthesizability linter

Static teaching checks from the HDL engine (`lintSynthesizability`): `#delay`, `initial`, `systasks`, `latch-risk` if/case, `blocking-in-FF`, and more. Not Yosys/Vivado — a lab-oriented rule pack. Starter: clean assign `y = a & b`.

Starter example: clean continuous assign. Try snippets that fail lint, then fix them.

Challenge 0 / 22 cleared

Quiz: what is this: This lab's `lintSynthesizability` API is...

☐ a teaching rule pack on the HDL AST — not full industrial synthesis

☐ identical to Vivado synth

☐ only a formatter

☐ a SPICE netlist checker

Show hint

Check

Next

Load challenge setup

Idle

Quiz: what is this

Quiz: #delay

Quiz: initial

Quiz: latch

Quiz: = in FF

Quiz: <= in comb

Quiz: \$display

Quiz: where lint lives

Lint: clean assign

Lint: catch #delay

Lint: catch initial

Lint: latch-risk

Synth lint lab starter

Real Verilog practice

```
wsl - module16-synth-lint/examples/navigation

# Track A - real Unix
# real Verilog session (Track A)

$ cat examples/synth_demo/synth_demo.v
// synth_demo.v - clean RTL vs common lint violations
module and2_clean(
    input a,
    input b,
    output y
);
    assign y = a & b;
endmodule

module ff_good(
    input      clk,
    input      rst_n,
    input      d,
    output reg  q
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) q ≤ 1'b0;
        else      q ≤ d;
    end
endmodule

// delay_bad - NOT synthesizable (commented)
// module delay_bad(input a, output y);
//     assign #5 y = a;
// endmodule

$ iverilog -t null examples/synth_demo/synth_demo.v
(syntax check passed)
```

Real Verilog - clean vs lint violations

Pitfalls to watch

- Do not treat a green simulation as proof of synthesizability
- Do not copy testbench patterns like dollar-display into RTL modules
- Do not mix blocking and non-blocking styles inside the same clocked always block
- Do not assume this teaching linter replaces industrial tools like Yosys or Vivado
- Run lint early and fix findings before you commit

Your turn

- Complete the checklist for at least one track, ideally both
- In the browser lab, load the latch snippet and name the rule that fires
- Any delays, initial blocks, or incomplete branches?
- When you finish, take the short quiz, then continue to HDL style in the next module